CINEMA BOOKING SYSTEM · FINAL REVIEW

CINEMA DBMS PROJECT

Schedule shows, book seats, sell concessions and issue tickets on a normalized relational database.

Name **Amaan Khan**
Class **AI/ML Leopards**
Roll **25WU0101011**

14 Tables

3NF Compliant

3 User roles

MOVIE	
movie_id	PK
title	
duration_min	

SCREENING	
screening_id	PK
movie_id	FK
start_time	

BOOKING	
booking_id	PK
customer_id	FK
total_amount	

SEAT	
seat_id	PK
screen_id	FK
seat_type	

AGENDA

WHAT THIS REVIEW COVERS

1 **PROBLEM AND OBJECTIVES**
Why bookings break

2 **TARGET USERS**
Who relies on it

3 **FUNCTIONAL REQUIREMENTS**
Scheduling, locking, transactions

4 **CONCEPTUAL DESIGN**
Two ER models, 14 tables

5 **SCHEMA AND 3NF**
Normalization and SQL constraints

6 **LIVE SIMULATION**
Booking, scheduling, refunds

7 **SALES, REPORTS AND TESTING**
Concessions, payments, 15 checks

8 **BRIEF AND BACKEND**
Coverage, API, future scope

PROBLEM AND OBJECTIVES

EXECUTIVE SUMMARY

Manual and loosely coupled systems in multi-screen cinemas lead to double-booked seats at peak hours, overlapping schedules on one screen, inconsistent pricing, and data errors that surface only after payment.

THESE FAILURES BECOME REVENUE LOSS, UNHAPPY CUSTOMERS AND RECONCILIATION WORK FOR STAFF.

OBJECTIVE

Design a relational schema, normalized to Third Normal Form, where the database itself enforces the business rules.

APPROACH

Constraints, keys and transactions do the policing, so no application bug can create a double booking or a bad refund.

ONE SEAT, ONE BOOKING

A seat can be assigned once per screening.

`UNIQUE (screening_id, seat_id)`

NO OVERLAPPING SHOWS

Two screenings never share a screen and time, with a cleaning buffer.

`EXCLUDE USING gist`

AUDITABLE REFUNDS

Cancellations follow refund limits and leave a log.

`CHECK + audit_log`

REFERENTIAL INTEGRITY

Bookings, customers and shows always point to real rows.

`FOREIGN KEY`

TARGET USERS AND ROLES

WHO USES THE SYSTEM

Each role touches a different set of tables, and nothing more.

CUSTOMERS

ONLINE

Browse showtimes, pick and book seats online, buy concessions in advance and view booking history.

CUSTOMERS READ THE SCHEDULE AND CREATE THEIR OWN BOOKINGS

`screening · SELECT` `seat · SELECT` `booking · INSERT` `booked_seat · INSERT`

```
-- upcoming shows
SELECT s.start_time, m.title, sc.name
FROM screening s
JOIN movie m USING (movie_id)
JOIN screen sc USING (screen_id)
WHERE s.start_time > now()
ORDER BY s.start_time;
```

BOX OFFICE

ON SITE

Handle walk-in sales, issue physical tickets, and process manual cancellations and exchanges.

BOX OFFICE STAFF SELL, CANCEL AND EXCHANGE

`booking · INSERT` `booked_seat · INSERT` `booking · UPDATE` `booked_seat · DELETE` `audit_log · INSERT`

```
-- walk-in sale, paid at the counter
INSERT INTO booking
  (customer_id, screening_id,
   total_amount, status)
VALUES (1, 101, 520.00, 'CONFIRMED')
RETURNING booking_id;
```

ADMINISTRATORS

BACK OFFICE

Manage the movie and pricing catalog, run occupancy and revenue reports, and audit transaction logs.

ADMINISTRATORS MANAGE THE CATALOG AND READ EVERY REPORT

`movie · INSERT` `screening · INSERT` `screening · UPDATE` `booking · SELECT` `audit_log · SELECT`

```
-- seats sold per movie
SELECT m.title, COUNT(*) AS seats_sold
FROM booked_seat bs
JOIN screening s USING (screening_id)
JOIN movie m USING (movie_id)
GROUP BY m.title
ORDER BY seats_sold DESC;
```

FUNCTIONAL REQUIREMENTS

THREE THINGS IT MUST DO WELL

DYNAMIC SHOW SCHEDULING

Allocate screens and time slots across 2D, 3D and IMAX without overlaps, including cleaning buffer time.

SIMULATION 2 · SCHEDULING

`EXCLUDE USING gist` `CHECK show_within_hours`

CONCURRENCY CONTROL

Seat locking so simultaneous users never collide. Conflicting transactions roll back safely.

SIMULATION 1 · BOOKING

`SELECT … FOR UPDATE`

`UNIQUE (screening_id, seat_id)`

TRANSACTIONAL PROCESSING

Ticket sales, concessions, coupons and refunds handled with full data consistency (ACID).

SIMULATION 3 · REFUNDS

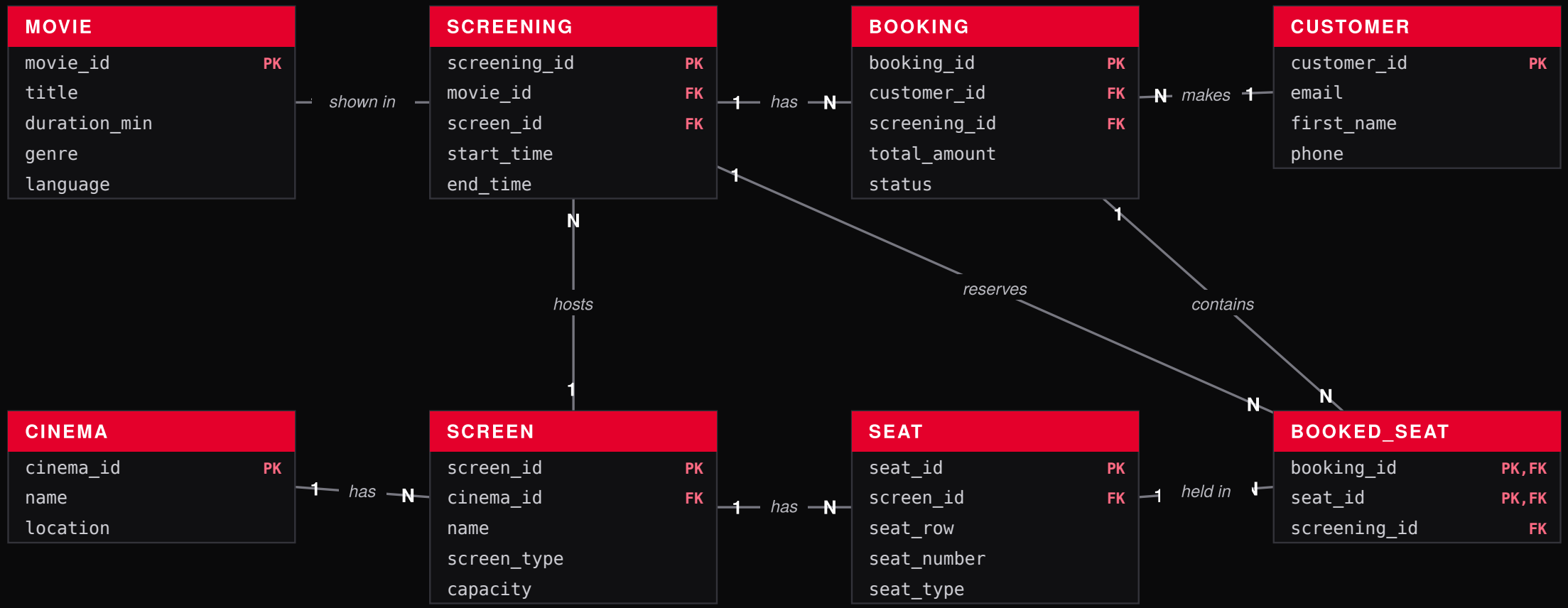
`BEGIN … COMMIT` `CHECK refund_within_total`

BUILT WITH POSTGRESQL · BTREE_GIST · ER MODELLING · 3NF · ACID TRANSACTIONS · ROW LOCKING · CHECK AND UNIQUE

CONCEPTUAL DESIGN

DATABASE ARCHITECTURE

Model 1 of 2. Bookings and seating: eight tables from cinema down to the seat held in each booking.

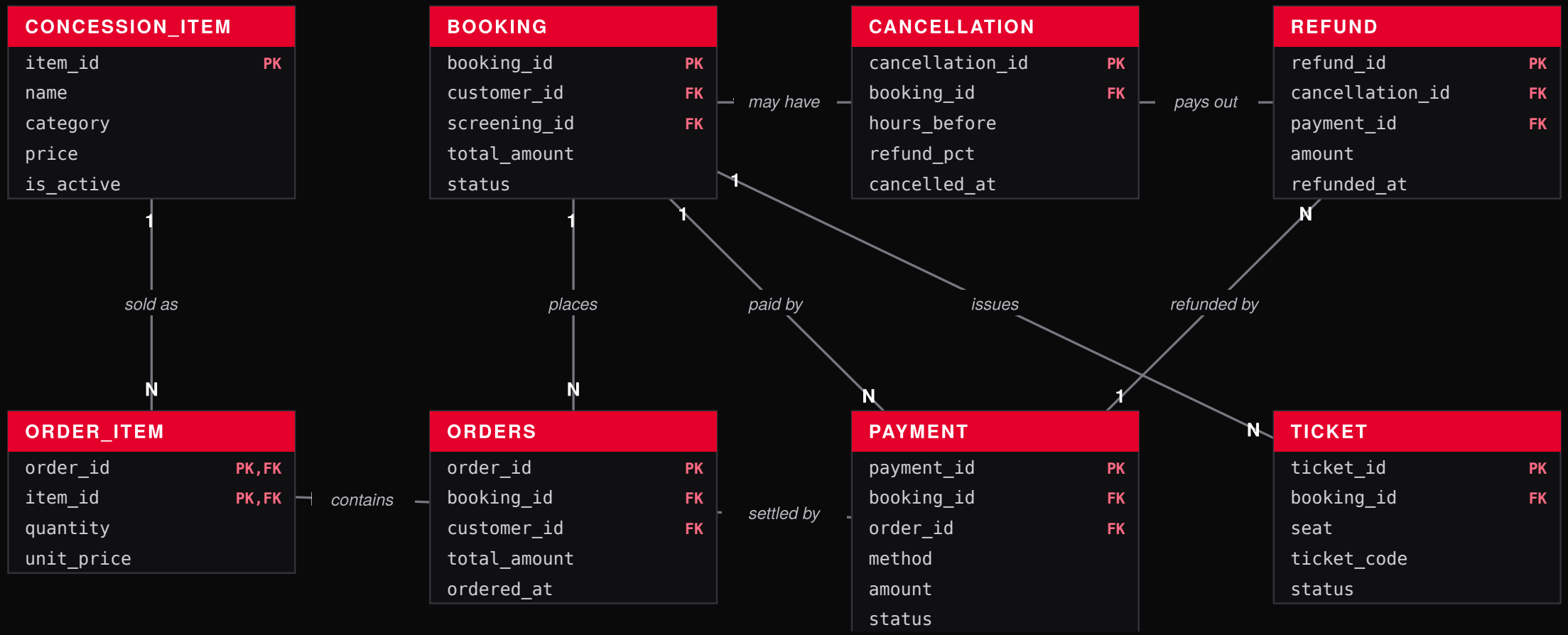


Core entity · Supporting table · **PK** primary key · **FK** foreign key · **1** one · **N** many

CONCEPTUAL DESIGN

SALES AND PAYMENTS

Model 2 of 2. Tickets, concessions, payments, cancellations and refunds. 14 tables in total. BOOKING is shared with model 1.



Core entity · Supporting table · PK primary key · FK foreign key · 1 one · N many

CONCEPTUAL DESIGN

RELATIONSHIPS SUMMARY

How the tables in model 2 connect, and the rule on each link.

RELATIONSHIP	RULE
booking → ticket	A booking issues one ticket per seat. Ticket code is UNIQUE.
booking → cancellation	A booking can be cancelled once. Policy: 100, 50 or 0 percent.
cancellation → refund	A cancellation produces at most one refund. Refund cannot exceed amount paid.
booking → orders	A booking can include concession orders. Counter sales have no booking.
booking → payment	A booking is paid by one or more payments. Amount must be greater than 0.
orders → order_item	An order contains one or more items. quantity must be greater than 0.
orders → payment	An order is settled by payments. Amount must be greater than 0.
payment → refund	A payment can be refunded. Refund limited to payment amount.
concession_item → order_item	A menu item appears in many orders. price must be greater than 0.

SCHEMA DESIGN

FROM ONE WIDE SHEET TO 3NF

UNNORMALIZED 1NF 2NF 3NF

One wide sheet. Seats sit in a list, and the customer, movie and screen details repeat on every booking.

booking_id	customer	email	phone	movie	duration	screen	type	start	seats	total
1001	Ananya Rao	ananya@mail.com	98xxxx0001	Midnight Orbit	148	Screen 1	2D	10:30	C4, C5	400
1002	Rohan Mehta	rohan@mail.com	98xxxx0002	Midnight Orbit	148	Screen 1	2D	10:30	D6, D7, D8	600
1003	Sara Iqbal	sara@mail.com	98xxxx0003	The Last Lighthouse	112	Screen 2	3D	11:00	B2, B3	560

Problems: a list in one cell, and details like Midnight Orbit 148 min stored again on every row.

STEP 1

UNNORMALIZED

One wide row per booking. Seats sit in a list, and customer, movie and screen details repeat on every booking.

STEP 2

1NF

Every cell holds one value. Each seat gets its own row, so the key becomes booking_id plus seat.

STEP 3

2NF

Nothing depends on part of a composite key. Seats get their own table.

STEP 4

3NF

No non-key column depends on another non-key column. Details live with their own keys.

SCHEMA DESIGN

FROM ONE WIDE SHEET TO 3NF

UNNORMALIZED 1NF 2NF 3NF

Every cell holds one value. Each seat gets its own row, so the key becomes booking_id plus seat.

booking_id	seat	customer	email	phone	movie	duration	screen	type	start	total
1001	C4	Ananya Rao	ananya@mail.com	98xxxx0001	Midnight Orbit	148	Screen 1	2D	10:30	400
1001	C5	Ananya Rao	ananya@mail.com	98xxxx0001	Midnight Orbit	148	Screen 1	2D	10:30	400
1002	D6	Rohan Mehta	rohan@mail.com	98xxxx0002	Midnight Orbit	148	Screen 1	2D	10:30	600
1002	D7	Rohan Mehta	rohan@mail.com	98xxxx0002	Midnight Orbit	148	Screen 1	2D	10:30	600
1002	D8	Rohan Mehta	rohan@mail.com	98xxxx0002	Midnight Orbit	148	Screen 1	2D	10:30	600
1003	B2	Sara Iqbal	sara@mail.com	98xxxx0003	The Last Lighthouse	112	Screen 2	3D	11:00	560
1003	B3	Sara Iqbal	sara@mail.com	98xxxx0003	The Last Lighthouse	112	Screen 2	3D	11:00	560

Still wrong: customer, movie and screen columns depend only on booking_id, which is part of the key.

STEP 1

UNNORMALIZED

One wide row per booking. Seats sit in a list, and customer, movie and screen details repeat on every booking.

STEP 2

1NF

Every cell holds one value. Each seat gets its own row, so the key becomes booking_id plus seat.

STEP 3

2NF

Nothing depends on part of a composite key. Seats get their own table.

STEP 4

3NF

No non-key column depends on another non-key column. Details live with their own keys.

SCHEMA DESIGN

FROM ONE WIDE SHEET TO 3NF

UNNORMALIZED 1NF 2NF 3NF

Seats move to their own table. The booking table now depends on its whole key.

BOOKING

booking_id PK	customer	email	phone	movie	duration	screen	type	start	total
1001	Ananya Rao	ananya@mail.com	98xxxx0001	Midnight Orbit	148	Screen 1	2D	10:30	400
1002	Rohan Mehta	rohan@mail.com	98xxxx0002	Midnight Orbit	148	Screen 1	2D	10:30	600
1003	Sara Iqbal	sara@mail.com	98xxxx0003	The Last Lighthouse	112	Screen 2	3D	11:00	560

BOOKED_SEAT

booking_id PK,FK	seat PK
1001	C4
1001	C5
1002	D6
1002	D7
1002	D8
1003	B2
1003	B3

Still wrong: duration depends on movie, and type depends on screen. Those are non-key columns depending on non-key columns.

STEP 1
UNNORMALIZED
One wide row per booking. Seats sit in a list, and customer, movie and screen details repeat on every booking.

STEP 2
1NF
Every cell holds one value. Each seat gets its own row, so the key becomes booking_id plus seat.

STEP 3
2NF
Nothing depends on part of a composite key. Seats get their own table.

STEP 4
3NF
No non-key column depends on another non-key column. Details live with their own keys.

SCHEMA DESIGN

FROM ONE WIDE SHEET TO 3NF

UNNORMALIZED 1NF 2NF 3NF

3NF result: no non-key column depends on another non-key column. Customer, movie and screen details live with their own keys.

CUSTOMER	
customer_id	PK
email	
first_name	
phone	

MOVIE	
movie_id	PK
title	
duration_min	
genre	

SCREEN	
screen_id	PK
name	
screen_type	
capacity	

SCREENING	
screening_id	PK
movie_id	FK
screen_id	FK
start_time	

BOOKING	
booking_id	PK
customer_id	FK
screening_id	FK
total_amount	

BOOKED_SEAT	
booking_id	PK,FK
seat_id	PK,FK
screening_id	FK

STEP 1

UNNORMALIZED

One wide row per booking. Seats sit in a list, and customer, movie and screen details repeat on every booking.

STEP 2

1NF

Every cell holds one value. Each seat gets its own row, so the key becomes booking_id plus seat.

STEP 3

2NF

Nothing depends on part of a composite key. Seats get their own table.

STEP 4

3NF

No non-key column depends on another non-key column. Details live with their own keys.

IMPLEMENTATION

SQL SCHEMA

PostgreSQL. The constraints on the next slides are what the simulations run against.

TABLES | NO OVERLAP | NO DOUBLE BOOKING | REFUND AND AUDIT | TICKETS AND PAYMENTS | INDEXES | VIEWS | QUERIES

```
CREATE TABLE cinema (
  cinema_id SERIAL PRIMARY KEY,
  name TEXT NOT NULL, location TEXT);
CREATE TABLE screen (
  screen_id SERIAL PRIMARY KEY,
  cinema_id INT NOT NULL REFERENCES cinema,
  name TEXT NOT NULL,
  screen_type TEXT
    CHECK (screen_type IN ('2D','3D','IMAX')),
  capacity INT CHECK (capacity > 0));
CREATE TABLE seat (
  seat_id SERIAL PRIMARY KEY,
  screen_id INT NOT NULL REFERENCES screen,
  seat_row CHAR(1), seat_number INT,
  seat_type TEXT,
  UNIQUE (screen_id, seat_row, seat_number));
```

```
CREATE TABLE movie (
  movie_id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  duration_min INT, genre TEXT,
  language TEXT);
CREATE TABLE customer (
  customer_id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  first_name TEXT, last_name TEXT,
  phone TEXT);
```

Eight tabs of SQL in this deck: tables, the three rules, tickets and payments, indexes, views and queries. The simulations run against these constraints.

IMPLEMENTATION

NO OVERLAP, NO DOUBLE BOOKING

RULE 02 · SCHEDULING

```
-- Rule 2: no two screenings overlap on one screen,
-- including a 20 minute cleaning buffer
CREATE EXTENSION btree_gist;
CREATE TABLE screening (
  screening_id SERIAL PRIMARY KEY,
  movie_id  INT NOT NULL REFERENCES movie,
  screen_id INT NOT NULL REFERENCES screen,
  start_time TIMESTAMPTZ NOT NULL,
  end_time   TIMESTAMPTZ NOT NULL,
  base_price NUMERIC(8,2),
  CHECK (end_time > start_time),
  CONSTRAINT screening_no_overlap
    EXCLUDE USING gist (
      screen_id WITH =,
      tstzrange(start_time,
        end_time + INTERVAL '20 minutes') WITH &&));
```

```
ERROR: conflicting key value violates exclusion constraint "screening_no_overlap"
```

RULE 01 · UNIQUENESS

```
-- Rule 1: a seat is assigned once per screening
CREATE TABLE booking (
  booking_id SERIAL PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customer,
  screening_id INT NOT NULL REFERENCES screening,
  booking_time TIMESTAMPTZ DEFAULT now(),
  total_amount NUMERIC(8,2) CHECK (total_amount >= 0),
  status TEXT CHECK (status IN ('CONFIRMED','CANCELLED')));
CREATE TABLE booked_seat (
  booking_id INT REFERENCES booking ON DELETE CASCADE,
  seat_id INT REFERENCES seat,
  screening_id INT NOT NULL REFERENCES screening,
  PRIMARY KEY (booking_id, seat_id),
  CONSTRAINT booked_seat_screening_seat_key
    UNIQUE (screening_id, seat_id));

-- Concurrency: lock the rows while booking
BEGIN;
SELECT * FROM screening
  WHERE screening_id = 101 FOR UPDATE;
-- insert booking + booked_seat rows, then
COMMIT;
```

```
ERROR: duplicate key violates unique constraint "booked_seat_screening_seat_key"
```

IMPLEMENTATION

REFUNDS, AUDIT AND INDEXES

RULE 03 · MONEY

```
-- Rule 3: refund limits and a permanent trail
ALTER TABLE booking
  ADD COLUMN cancelled_at TIMESTAMPTZ,
  ADD COLUMN refund_amount NUMERIC(8,2) DEFAULT 0,
  ADD CONSTRAINT refund_within_total
    CHECK (refund_amount >= 0
       AND refund_amount <= total_amount),
  ADD CONSTRAINT cancel_once
    CHECK ((status = 'CANCELLED')
         = (cancelled_at IS NOT NULL));
CREATE TABLE audit_log (
  log_id BIGSERIAL PRIMARY KEY,
  booking_id INT REFERENCES booking,
  action TEXT NOT NULL,
  detail TEXT,
  logged_at TIMESTAMPTZ DEFAULT now());
```

REFUND AND AUDIT

WHAT THE DATABASE GUARANTEES

A refund can never be negative or above the booking total. A booking is cancelled only once, because cancelled_at must be set exactly when the status is CANCELLED. Every action leaves a row in audit_log.

IMPLEMENTATION

INDEXES ON SEARCHED COLUMNS


```
-- Indexes on frequently searched attributes
CREATE INDEX idx_screening_screen_time
  ON screening (screen_id, start_time);
CREATE INDEX idx_screening_movie
  ON screening (movie_id);
CREATE INDEX idx_booking_customer
  ON booking (customer_id);
CREATE INDEX idx_booking_screening
  ON booking (screening_id, status);
CREATE INDEX idx_payment_booking
  ON payment (booking_id);
CREATE INDEX idx_orders_booking
  ON orders (booking_id);
CREATE UNIQUE INDEX idx_customer_email
  ON customer (lower(email));
```

```
-- booked_seat (screening_id, seat_id) is already
-- indexed by its UNIQUE constraint, so seat
-- availability for one show is an index lookup:
EXPLAIN ANALYZE
SELECT seat_id FROM booked_seat
WHERE screening_id = 101;

-- Index Scan using booked_seat_screening_seat_key
```

IMPLEMENTATION

TICKETS AND PAYMENTS


```
CREATE TABLE ticket (
  ticket_id SERIAL PRIMARY KEY,
  booking_id INT NOT NULL,
  seat_id INT NOT NULL,
  screening_id INT NOT NULL REFERENCES screening,
  ticket_code TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'VALID'
    CHECK (status IN ('VALID','VOID','USED')),
  FOREIGN KEY (booking_id, seat_id)
    REFERENCES booked_seat);
CREATE TABLE concession_item (
  item_id SERIAL PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  category TEXT NOT NULL
    CHECK (category IN ('Snacks','Drinks','Combos')),
  price NUMERIC(8,2) NOT NULL CHECK (price > 0),
  is_active BOOLEAN NOT NULL DEFAULT TRUE);
CREATE TABLE orders (
  order_id SERIAL PRIMARY KEY,
  booking_id INT REFERENCES booking, -- NULL: counter sale
  customer_id INT NOT NULL REFERENCES customer,
  total_amount NUMERIC(8,2) NOT NULL
    CHECK (total_amount >= 0),
  ordered_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE TABLE order_item (
  order_id INT REFERENCES orders ON DELETE CASCADE,
  item_id INT REFERENCES concession_item,
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(8,2) NOT NULL CHECK (unit_price > 0),
  PRIMARY KEY (order_id, item_id));
```

```
CREATE TABLE payment (
  payment_id SERIAL PRIMARY KEY,
  booking_id INT REFERENCES booking,
  order_id INT REFERENCES orders,
  method TEXT NOT NULL
    CHECK (method IN ('UPI','Card','Cash','Net Banking')),
  amount NUMERIC(8,2) NOT NULL CHECK (amount > 0),
  status TEXT NOT NULL DEFAULT 'PAID'
    CHECK (status IN ('PAID','PART REFUNDED','REFUNDED')),
  paid_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (booking_id IS NOT NULL OR order_id IS NOT NULL));
CREATE TABLE cancellation (
  cancellation_id SERIAL PRIMARY KEY,
  booking_id INT NOT NULL UNIQUE REFERENCES booking,
  cancelled_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  hours_before NUMERIC(5,1) NOT NULL,
  refund_pct INT NOT NULL CHECK (refund_pct IN (0, 50, 100)));
CREATE TABLE refund (
  refund_id SERIAL PRIMARY KEY,
  cancellation_id INT NOT NULL UNIQUE REFERENCES cancellation,
  payment_id INT NOT NULL REFERENCES payment,
  amount NUMERIC(8,2) NOT NULL CHECK (amount > 0),
  refunded_at TIMESTAMPTZ NOT NULL DEFAULT now());
```

IMPLEMENTATION

TICKETS AND PAYMENTS: REFUND LIMIT


```
-- Refund limit: a refund can never exceed what was paid
CREATE FUNCTION check_refund_limit() RETURNS trigger AS $$
BEGIN
  IF NEW.amount >
     (SELECT amount FROM payment
      WHERE payment_id = NEW.payment_id) THEN
    RAISE EXCEPTION 'refund exceeds amount paid';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER refund_limit BEFORE INSERT ON refund
  FOR EACH ROW EXECUTE FUNCTION check_refund_limit();
```

WHY A TRIGGER

A CHECK constraint can only look at its own row. The refund amount must be compared with a row in another table (payment), so a BEFORE INSERT trigger does it. The booking table also carries CHECK refund_within_total as a second guard.

IMPLEMENTATION

VIEWS FOR REPORTS


```sql
CREATE VIEW v_show_occupancy AS
SELECT s.screening_id, m.title, sc.name AS screen,
       COUNT(bs.seat_id) AS seats_sold, sc.capacity,
       ROUND(100.0 * COUNT(bs.seat_id) / sc.capacity, 1)
         AS occupancy_pct
FROM screening s
JOIN movie m USING (movie_id)
JOIN screen sc USING (screen_id)
LEFT JOIN booked_seat bs USING (screening_id)
GROUP BY s.screening_id, m.title, sc.name, sc.capacity;
```

```sql
CREATE VIEW v_movie_performance AS
SELECT m.title,
       COUNT(DISTINCT s.screening_id) AS shows,
       COALESCE(SUM(b.total_amount), 0) AS ticket_revenue
FROM movie m
LEFT JOIN screening s ON s.movie_id = m.movie_id
LEFT JOIN booking b ON b.screening_id = s.screening_id
                   AND b.status = 'CONFIRMED'
GROUP BY m.movie_id;

CREATE VIEW v_concession_sales AS
SELECT ci.name, ci.category,
       SUM(oi.quantity) AS qty_sold,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM concession_item ci
JOIN order_item oi ON oi.item_id = ci.item_id
GROUP BY ci.item_id;
```

IMPLEMENTATION

QUERIES


```sql
-- 1. JOIN: every booking with customer, movie, screen
SELECT b.booking_id, cu.first_name, m.title,
       sc.name AS screen, b.total_amount, b.status
FROM booking b
JOIN customer cu ON cu.customer_id = b.customer_id
JOIN screening s ON s.screening_id = b.screening_id
JOIN movie m     ON m.movie_id = s.movie_id
JOIN screen sc   ON sc.screen_id = s.screen_id;

-- 2. Aggregate: seats and revenue per movie
SELECT m.title, COUNT(bs.seat_id) AS seats_sold,
       SUM(b.total_amount) AS revenue
FROM movie m
JOIN screening s ON s.movie_id = m.movie_id
JOIN booking b ON b.screening_id = s.screening_id
              AND b.status = 'CONFIRMED'
JOIN booked_seat bs ON bs.booking_id = b.booking_id
GROUP BY m.title ORDER BY revenue DESC;
```

```sql
-- 3. Nested: bookings above the average value
SELECT booking_id, total_amount FROM booking
WHERE total_amount >
  (SELECT AVG(total_amount) FROM booking
   WHERE status = 'CONFIRMED');

-- 4. Nested: movies that have never been booked
SELECT title FROM movie
WHERE movie_id NOT IN
  (SELECT s.movie_id FROM screening s
   JOIN booking b ON b.screening_id = s.screening_id);

-- 5. Concession sales with HAVING
SELECT ci.name, SUM(oi.quantity) AS qty,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM concession_item ci
JOIN order_item oi ON oi.item_id = ci.item_id
GROUP BY ci.name HAVING SUM(oi.quantity) > 1
ORDER BY revenue DESC;
```

IMPLEMENTATION

RULES THE DATABASE ENFORCES

BUSINESS RULE	TYPE	HOW THE DATABASE ENFORCES IT
One seat per screening	UNIQUE	UNIQUE (screening_id, seat_id) on booked_seat
No screen overlap, 20 minute gap	EXCLUDE	screening_no_overlap (EXCLUDE in PostgreSQL, trigger in SQLite)
Shows run 10:00 to midnight	CHECK	CHECK constraint show_within_hours
Positive quantity and price	CHECK	quantity_positive, price_positive
Valid cancellation policy (100 / 50 / 0%)	CHECK	CHECK (refund_pct IN (0, 50, 100))
Cancel only once	UNIQUE	cancel_once trigger and UNIQUE on cancellation.booking_id
Refund never above the amount paid	TRIGGER	refund_limit trigger and CHECK refund_within_total on booking
No deleting a show that has bookings	FOREIGN KEY	Foreign keys from booking to screening

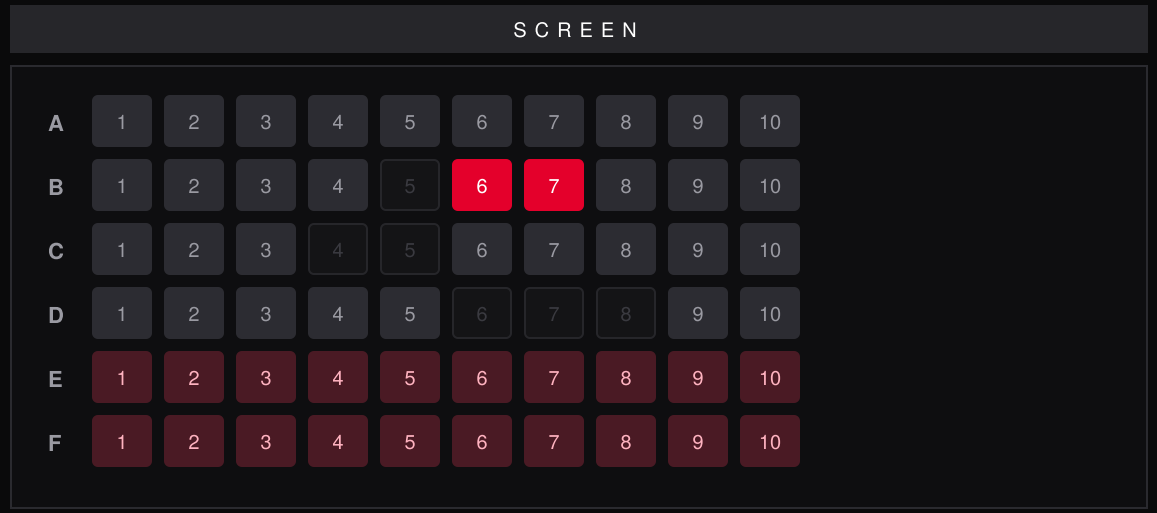
THE APPLICATION CANNOT BREAK THESE RULES, EVEN BY MISTAKE.

LIVE SIMULATION 1 OF 3

BOOKING AND SEAT LOCKING

Pick seats and pay. A second customer wins one seat, so the first transaction rolls back and nothing is saved.

SCREENING · MIDNIGHT ORBIT · SCREEN 1 · 10:30



Seat B5 was taken by another customer a moment earlier, so it shows as booked. Standard seats cost ₹200 on Screen 1, and rows E and F add ₹60.

First attempt: B5, B6 · Ananya Rao · ₹400 · UPI. Another customer grabs the first seat at the same moment, so this transaction rolls back and nothing is saved.

DB CONSOLE

```
BEGIN;
SELECT * FROM screening WHERE screening_id = 101 FOR UPDATE;
-- [B Rohan Mehta] COMMIT;  -- took seat B5 first
INSERT INTO booked_seat ... seat B5;
ERROR: duplicate key violates unique constraint
       "booked_seat_screening_seat_key"
ROLLBACK;  -- booking, payment and concessions not saved

BEGIN;     -- retry with B6, B7
SELECT * FROM screening WHERE screening_id = 101 FOR UPDATE;
INSERT INTO booking (customer_id, screening_id) VALUES (1, 101);
INSERT INTO booked_seat ...;  -- B6, B7
INSERT INTO ticket ...;       -- TK-1005-B6, TK-1005-B7
INSERT INTO payment (booking_id, method, amount)
  VALUES (1005, 'UPI', 400);
COMMIT;    -- booking #1005 confirmed
```

CINEMA DBMS · ADMIT 2

MIDNIGHT ORBIT

Screen 1 · 2D · 10:30 · Seats B6, B7

TK-1005-B6 · TK-1005-B7

#1005 · ₹400 · confirmed

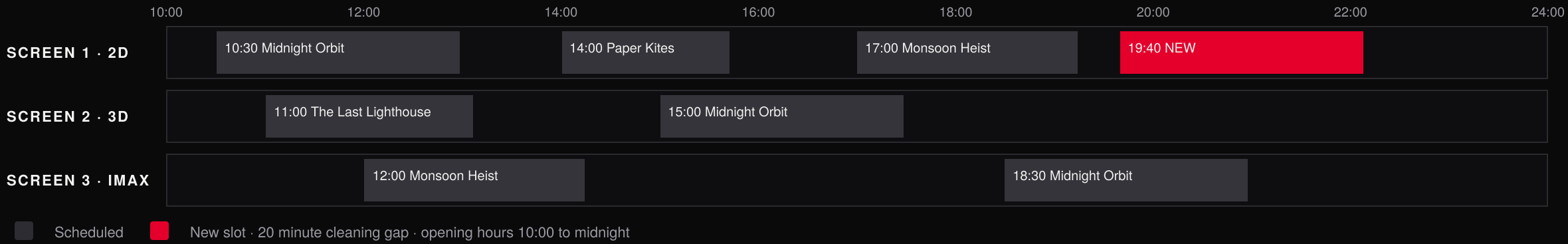
LIVE SIMULATION 2 OF 3

SCHEDULING WITHOUT OVERLAPS

Plan a show for Mon 12 Oct. Every show needs a 20 minute cleaning gap, and shows run from 10:00 to midnight.

Screen 1 · 2D · Midnight Orbit (148 min) · start 11:00 AM

```
ERROR: conflicting key value violates exclusion constraint "screening_no_overlap". Existing: Midnight Orbit, 10:30 to 12:58 plus the 20 minute gap.
```



11:00 BLOCKED

Overlaps Midnight Orbit (10:30 to 12:58) on Screen 1

13:10 BLOCKED

Starts inside the 20 minute gap after 12:58

19:40 ADDED

Fits after Monsoon Heist plus the gap: INSERT 0 1

23:00 BLOCKED

Would run past midnight: CHECK show_within_hours

LIVE SIMULATION 3 OF 3

CANCELLATIONS, REFUNDS AND AUDIT

₹280	1	₹280	33%
Total refunded	Cancellations	Revenue kept	Cancellation rate

REFUND POLICY · HOURS UNTIL THE SHOW: 2

OVER 24 H · 100%	**2 TO 24 H · 50%**	UNDER 2 H · 0%

A refund can never exceed the booking total, and a booking can be cancelled only once.

50%
2 h before

BOOKINGS

#1001	Ananya Rao C4, C5	Midnight Orbit Screen 1 · 10:30	₹400	CONFIRMED
#1002	Rohan Mehta D6, D7, D8	Midnight Orbit Screen 1 · 10:30	₹600	CONFIRMED
#1003	Sara Iqbal B2, B3	The Last Lighthouse Screen 2 · 11:00	₹560 · Refund ₹280	CANCELLED

DB CONSOLE

```
BEGIN;
UPDATE booking SET status='CANCELLED', refund_amount=280
  WHERE booking_id=1003;
DELETE FROM booked_seat WHERE booking_id=1003;
COMMIT;
```

AUDIT LOG AND RECORDS

```
CANCELLED #1003 · refund ₹280 (50%) · seats released
cancellation #7001 · 2 h before · 50% refund
refund #8001 · payment #9003 · ₹280
```

Booking #1003 cancelled. Refund ₹280 of ₹560 (50%). Seats are free again.

BEYOND BOOKINGS

CONCESSIONS, PAYMENTS AND TICKETS

CONCESSIONS

Six menu items. Orders hang off a booking, or stand alone for counter sales. Quantity and price must be positive.

Popcorn (Regular)	SNACKS	₹150
Popcorn (Large)	SNACKS	₹250
Nachos and Cheese	SNACKS	₹200
Soft Drink	DRINKS	₹120
Cold Coffee	DRINKS	₹140
Combo: Popcorn + 2 Drinks	COMBOS	₹420

Order #5001 · booking #1001
2 × Popcorn (Regular) + 2 × Soft Drink = ₹540

PAYMENTS

Every booking and order is paid through one payment record, by UPI, card, cash or net banking.

UPI	₹940
Card	₹600
Cash	₹560
Net Banking	₹0

₹2,100
Total received

₹2,100
Net of refunds

CHECK amount > 0 and a payment must point at a booking or an order.

TICKETS

One ticket per seat, with a unique code. A ticket points at the exact booked seat, so a seat can never be ticketed twice.

CINEMA DBMS · ADMIT 2

MIDNIGHT ORBIT

Screen 1 · 2D · 10:30 · Seats C4, C5

`TK-1001-C4 · TK-1001-C5`

#1001 · ₹400

```
ticket_code TEXT UNIQUE
FOREIGN KEY (booking_id, seat_id)
  → booked_seat
status IN (VALID, VOID, USED)
```

REPORTS

LIVE QUERIES ON LIVE DATA

SHOW OCCUPANCY | MOVIE PERFORMANCE | SCREEN UTILIZATION | CONCESSION SALES | CANCELLATIONS | REVENUE

```
CREATE VIEW v_show_occupancy AS
SELECT s.screening_id, m.title, sc.name AS screen,
       COUNT(bs.seat_id) AS seats_sold, sc.capacity,
       ROUND(100.0 * COUNT(bs.seat_id) / sc.capacity, 1)
         AS occupancy_pct
FROM screening s
JOIN movie m USING (movie_id)
JOIN screen sc USING (screen_id)
LEFT JOIN booked_seat bs USING (screening_id)
GROUP BY s.screening_id, m.title, sc.name, sc.capacity;
```

SHOW	SCREEN	SEATS SOLD	OCCUPANCY	TICKET REVENUE
Midnight Orbit · 10:30	Screen 1	5 / 60	8%	₹1,000
The Last Lighthouse · 11:00	Screen 2	2 / 60	3%	₹560
Monsoon Heist · 12:00	Screen 3	0 / 60	0%	₹0
Paper Kites · 14:00	Screen 1	0 / 60	0%	₹0
Midnight Orbit · 15:00	Screen 2	0 / 60	0%	₹0
Monsoon Heist · 17:00	Screen 1	0 / 60	0%	₹0
Midnight Orbit · 18:30	Screen 3	0 / 60	0%	₹0

₹1,560
Revenue

3
Confirmed bookings

7
Seats sold

2%
Average occupancy

Six reports read the same live data as the simulations: occupancy, movie performance, screen utilization, concession sales, cancellations and revenue. Book or cancel in the app and the numbers change.

REPORTS

MOVIE PERFORMANCE AND SCREEN UTILIZATION

MOVIE PERFORMANCE

```
SELECT * FROM v_movie_performance
ORDER BY ticket_revenue DESC;
```

MOVIE	SHOWS	TICKET REVENUE
Midnight Orbit	3	₹1,000
The Last Lighthouse	1	₹560
Monsoon Heist	2	₹0
Paper Kites	1	₹0

SCREEN UTILIZATION

```
-- scheduled minutes out of 840 open minutes
SELECT sc.name, COUNT(s.screening_id) AS shows,
  COALESCE(SUM(s.end_min - s.start_min), 0) AS minutes,
  ROUND(100.0 * COALESCE(SUM(s.end_min - s.start_min),0)
        / 840, 1) AS utilization_pct
FROM screen sc
LEFT JOIN screening s ON s.screen_id = sc.screen_id
GROUP BY sc.screen_id;
```

SCREEN	SHOWS	MINUTES	UTILIZATION
Screen 1	3	384	45.7%
Screen 2	2	274	32.6%
Screen 3	2	282	33.6%

REPORTS

CONCESSIONS, CANCELLATIONS AND REVENUE

CONCESSION SALES

```
SELECT * FROM v_concession_sales;
```

ITEM	CATEGORY	QTY	REVENUE
Popcorn (Regular)	Snacks	2	₹300
Soft Drink	Drinks	2	₹240

CANCELLATIONS

```
SELECT c.booking_id, c.hours_before,
       c.refund_pct, r.amount AS refunded
FROM cancellation c
LEFT JOIN refund r
  ON r.cancellation_id = c.cancellation_id;
```

BOOKING	HOURS BEFORE	REFUND	REFUNDED
#1003	2.0	50%	₹280

REVENUE

```
SELECT 'Tickets', SUM(total_amount)
  FROM booking WHERE status='CONFIRMED'
UNION ALL SELECT 'Concessions', SUM(total_amount)
  FROM orders
UNION ALL SELECT 'Refunds', -COALESCE(SUM(amount),0)
  FROM refund;
```

SOURCE	AMOUNT
Tickets	₹1,560
Concessions	₹540
Refunds	-₹280

TESTING

DO THE RULES REALLY HOLD?

15

OF 15 TESTS PASSED

WHAT IS CHECKED

Each check runs against the same logic and data that power the simulations: scheduling gaps, the refund policy, seat prices, midnight cutoff, concession rules, payments and the live tables. Run them from Administrator, Tests.

A show starting inside the 20 minute gap is blocked	PASS
A show starting after the gap is allowed	PASS
24 hours or more before the show refunds 100%	PASS
2 to 24 hours before the show refunds 50%	PASS
Under 2 hours before the show refunds 0%	PASS
Refund never exceeds the booking total	PASS
Premium rows E and F cost 60 rupees more	PASS
A show running past midnight is rejected	PASS
Live data: every confirmed seat is held by exactly one booking	PASS
Live data: no two shows overlap on one screen	PASS
Concession quantity must be a whole number greater than 0	PASS
Concession and menu prices are positive	PASS
Live data: every payment equals its booking or order total	PASS
Live data: refunds never exceed the amount paid	PASS
Live data: a booking is cancelled at most once	PASS

PROJECT BRIEF

WHAT THE BRIEF ASKED FOR

REQUIREMENT	WHERE IT IS DONE
Entities: cinemas, screens, seats, movies, screenings, customers, bookings, booked seats, tickets, concessions, orders, payments, cancellations, refunds	Two ER diagrams: the booking model, and the sales and payments model
ER model converted to a relational schema	SQL schema: Tables and Tickets and payments tabs
Normalized to Third Normal Form	Normalization walkthrough: 1NF, 2NF, 3NF
PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT	Every table uses all six
One seat per screening	UNIQUE (screening_id, seat_id), tested by the seat clash
No screen overlap	EXCLUDE with a 20 minute cleaning gap, tested on Scheduling
Positive quantity	CHECK (quantity > 0), enforced at checkout
Valid cancellation policy and refund limits	100, 50 or 0 percent by hours before the show; refund never above amount paid
Realistic sample data	Database page: 14 tables with movies, shows, bookings, orders and payments
Retrieval, joins, aggregates, nested queries, views	SQL schema: Queries and Views tabs
Reports: occupancy, movie performance, screen utilization, concessions, cancellations, revenue	Administrator, Reports: six reports with their SQL
Front-end for scheduling, availability, booking, cancellation, tickets, concessions, payment, reporting	Simulation app, and a Python backend with a real database file
Transactions, indexing, input validation, systematic testing	DB console BEGIN/COMMIT, Indexes tab, form validation, 15 tests

PROJECT PACKAGE

BACKEND AND DATABASE FILE

PROJECT PACKAGE

FILE	WHAT IT IS
server.py	Python backend, standard library only
cinema.db	SQLite database file with 14 tables and sample data
sql/schema.sql	Tables, keys, rules, triggers, indexes, views
sql/seed.sql	Realistic sample data
sql/queries.sql	Joins, aggregates, nested queries, six reports
sql/schema_postgres.sql	The same design written for PostgreSQL
public/index.html	Website and simulation, connected to the backend
standalone/index.html	The same site with no backend

API

	ENDPOINT	WHAT IT DOES
GET	/api/state	All tables
POST	/api/checkout	Seats and concessions, tickets and payment in one transaction
POST	/api/bookings/<id>/cancel	Cancel with refund, cancellation and refund records
POST	/api/movies	Add a movie
POST	/api/menu	Add a concession item
POST	/api/screenings	Schedule a show
DELETE	/api/screenings/<id>	Delete a show
POST	/api/reset	Restore sample data

```
# run with the backend: every booking, order, payment, refund and screening is saved in cinema.db
python3 server.py        # then open http://localhost:8000 and launch the simulation
sqlite3 -header -column cinema.db "SELECT * FROM v_show_occupancy;"
```

BEYOND THE REVIEW

FUTURE SCOPE

PAYMENT GATEWAY

Link real payments to bookings with a payment table and status checks.

CONCESSIONS AND COUPONS

Add snacks, combo offers and discount codes inside the same transaction.

DYNAMIC PRICING

Prices that change by time, weekday and demand.

QR TICKETS

Mobile tickets scanned at the door and marked as used.

INDEXES AND PARTITIONS

Keep queries fast as years of bookings pile up.

ADMIN DASHBOARD

Charts for occupancy, revenue and refunds from the report queries.

THANK YOU

Normalized to 3NF, with seat conflicts, show overlaps and refund limits enforced by the database.

14	3NF	4
Tables	Compliant	Rules enforced